CRYPTOGRAPHY

& INFORMATION SYSTEM SECURITY

Modul praktikum:
Kriptografi Modern (Kunci Simetris)

Tim Asisten Praktikum

2026

Daftar Isi

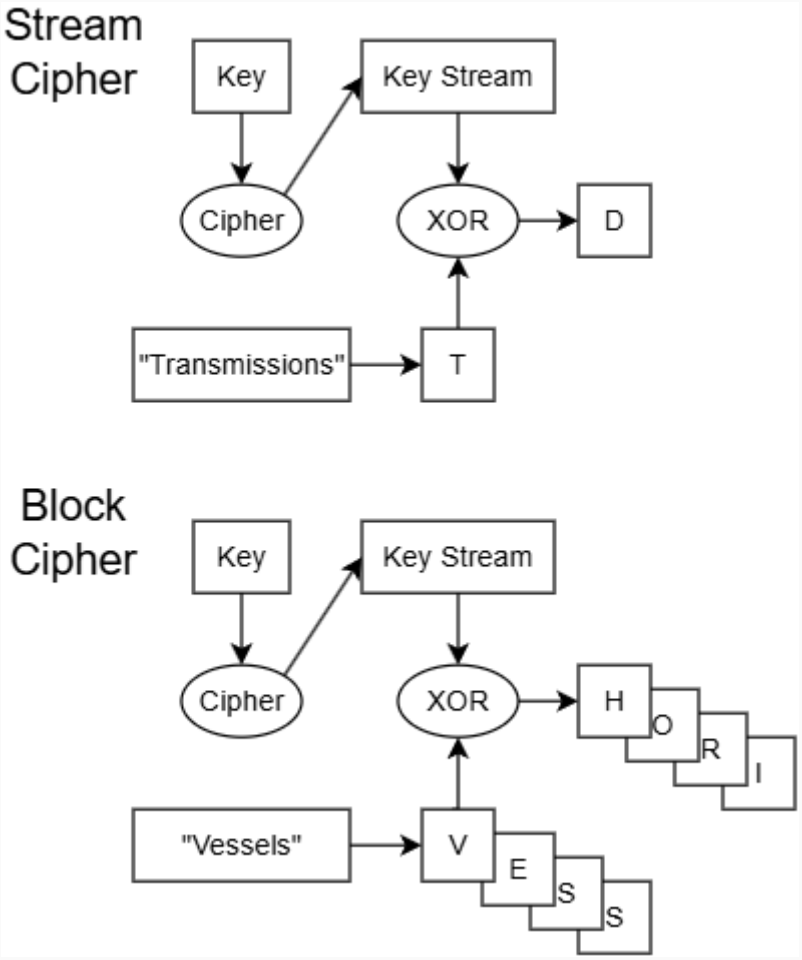
Daftar Isi **2**
Predecessor **3**
Stream Cipher 3
Block Cipher 5
Symmetric Cipher Model 6
AES **7**
Overview 7
Implementasi 8
Tugas **10**
Daftar Pustaka **11**
Tim Asisten Praktikum **11**

Predecessor

Stream Cipher

Stream cipher adalah pendekatan enkripsi terhadap aliran data digital, satu bit ataupun satu byte, dalam satu waktu (Stallings, 2023). Berbeda dengan kriptografi klasik yang enkripsinya dilakukan per karakter, kriptografi modern melakukannya dengan pendekatan stream cipher ataupun block cipher.

Contoh sederhana: Jika plaintext adalah “HALO” (4 byte), Stream Cipher akan mengenkripsi H, kemudian A, kemudian L, kemudian O secara terpisah. Sedangkan Block Cipher akan mengambil keempat byte tersebut (ditambah padding jika perlu hingga memenuhi ukuran blok, misalnya 16 byte) dan memprosesnya sebagai satu kesatuan.



Gambar 1. *Perbandingan Stream dengan Block Cipher*
Sumber: Yours truly

Sebelum mengenal bagaimana Stream Cipher bekerja, cara mengolah data bit per bit ditunjukkan pada Algoritma 1.

Algoritma 1. *Implementasi Pembacaan String sebagai Bit*

```
def baca_bit(teks):
    for c in teks:
        # Kita tahu 1 char = 8 bit
```

```
        # Kita gunakan loop dari bit paling signifikan (MSB) ke paling rendah (LSB)
        for i in range(7, -1, -1):
            # Operasi Bitwise:
            # 1. (ord(c) >> i) menggeser bit ke posisi paling kanan
            # 2. & 1 membandingkan bit tersebut dengan 1 (masking)
            bit = (ord(c) >> i) & 1
            print(bit, end='')
        print(' ', end='')  # Spasi antar karakter agar mudah dibaca
    print()

if __name__ == "__main__":
    input_text = "saya"
    print(f"Teks: {input_text}")
    print("Bit : ", end='')
    baca_bit(input_text)
```

Melalui contoh di atas, pembacaan per byte ditunjukkan pada Algoritma 2.

Algoritma 2. *Function Pembacaan String sebagai Byte*

```
def proses_dengan_bitset(teks):
    for c in teks:
        # Mengubah 1 byte (char) menjadi representasi biner 8-bit
        b = format(ord(c), '08b')
        print(f"Karakter [{c}] biner: {b}")

        # Kamu bisa akses bit spesifik seperti string
        # b[-1] adalah bit paling kanan (LSB)
        # b[0] adalah bit paling kiri (MSB)
        if b[-1] == '1':
            # Sama dengan mengecek apakah bit ke-0 adalah 1
            pass
```

Pada praktiknya, enkripsi tidak terbatas pada string. Salah satu contohnya adalah file digital. Maka dari itu, pembacaan pun dilakukan dalam bentuk byte. Contoh implementasi ditunjukkan pada Algoritma 3.

Algoritma 3. *Imlementasi Stream Cipher*

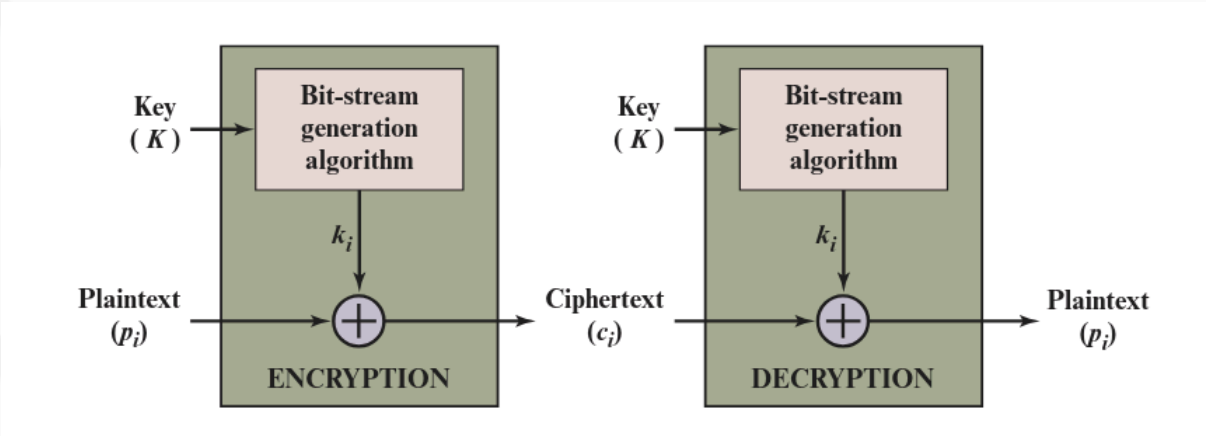
```
def xor_file(input_file, output_file, key):
    # Membuka file input dalam mode biner
    try:
        with open(input_file, 'rb') as fin:
            data = fin.read()
    except FileNotFoundError:
        print("Gagal membuka file!")
        return

    # Operasi XOR pada setiap byte
    result = bytes([b ^ key for b in data])

    # Menulis byte hasil XOR ke file tujuan
    with open(output_file, 'wb') as fout:
        fout.write(result)

    print(f"Proses selesai. File disimpan di: {output_file}")

if __name__ == "__main__":
    file_asal = "pesan.txt"
    file_hasil = "pesan_enkripsi.dat"
    kunci = 0x42  # Kunci bebas (1 byte)
```

```
    # Contoh penggunaan
    xor_file(file_asal, file_hasil, kunci)
```

Untuk menggunakannya, buatlah file *pesan.txt* berisi kalimat `percobaan stream cipher`. Pastikan hasil *pesan_enkripsi.dat* selalu sama untuk setiap orang. Cobalah ganti *pesan.txt* dengan file image atau apapun selain extension txt. Untuk membaca perubahan file, dapat digunakan aplikasi pembaca file dalam hexadecimal.

1. Unduh HexEdit!
2. Buka file image yang belum dan sudah dienkripsi!

Perhatikan bahwa ada operasi XOR pada Implementasi Stream Cipher. Secara sederhana, model stream digambarkan pada Gambar 2.



Gambar 2. *Stream Cipher*
Sumber: Stallings, 2023

Block Cipher

Jika Stream Cipher mengolah data dalam bentuk bit ataupun byte, Block Cipher mengolahnya dalam bentuk kumpulan byte. Dengan kata lain, Block Cipher adalah pendekatan dimana block, kumpulan byte plaintext, diperlakukan secara keseluruhan dan menghasilkan block dengan panjang yang sama(Stallings, 2023). Algoritma 4 adalah contoh implementasi dengan ukuran block sebesar 128 bits.

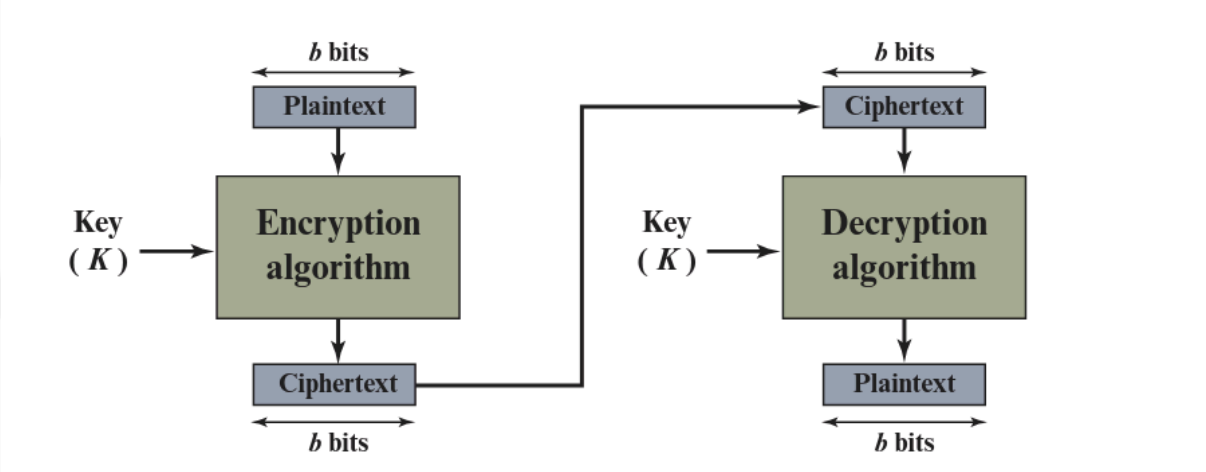
Contoh: Pada Algoritma 4, ukuran blok adalah 128 bits (16 bytes). Artinya, setiap kali program membaca file, ia mengambil 16 byte sekaligus, melakukan operasi XOR terhadap 16 byte tersebut dengan kunci 16 byte, lalu menulis hasilnya. Jika sisa data kurang dari 16 byte, maka hanya byte yang tersisa yang diproses.

Algoritma 4. *Implementasi Block Cipher*

```
def xor_file_128bit(input_file, output_file, key_16bytes):
    try:
        with open(input_file, 'rb') as fin:
            data = fin.read()
    except FileNotFoundError:
        print("Gagal membuka file!")
        return

    # 128 bits = 16 bytes
    block_size = 16
    result = bytearray()
```

```
    for i in range(0, len(data), block_size):
        block = data[i:i+block_size]
        # Proses XOR blok 128-bit dengan kunci 128-bit
        xor_block = bytes([
            b ^ key_16bytes[j % len(key_16bytes)]
            for j, b in enumerate(block)
        ])
        result.extend(xor_block)

    with open(output_file, 'wb') as fout:
        fout.write(result)

    print("Proses blok 128-bit selesai!")

if __name__ == "__main__":
    # Kunci 128-bit (16 karakter/byte)
    key_str = "HIDUPJOKOWI12345"
    key = key_str.encode('utf-8')

    xor_file_128bit("gambar.jpg", "gambar_aman.dat", key)
```

Seperti halnya Stream Cipher, model Block Cipher ditunjukkan pada Gambar 3.

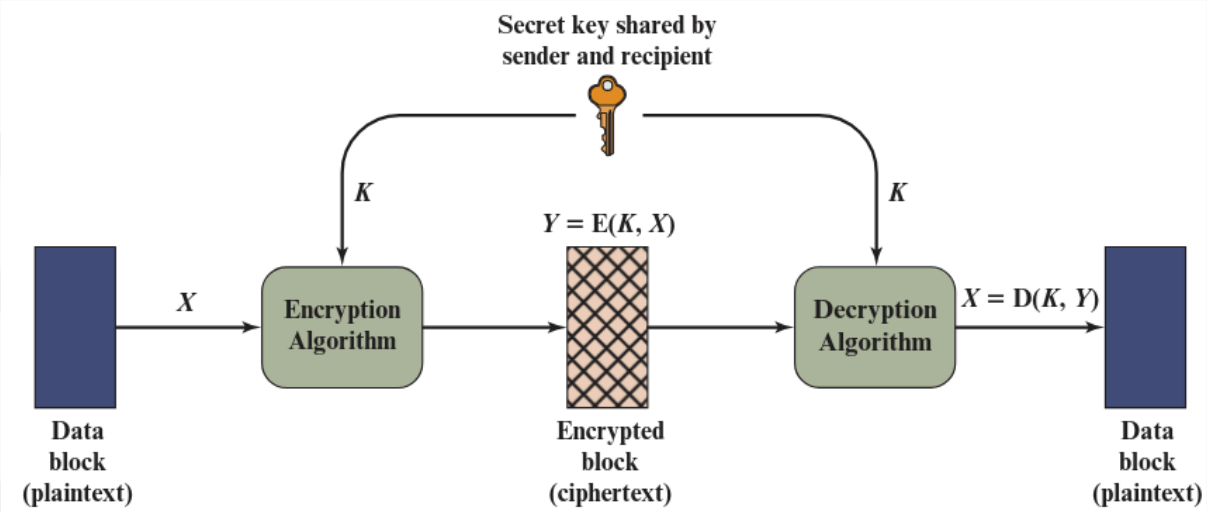


Gambar 3. *Block Cipher*
Sumber: Stallings, 2023

Symmetric Cipher Model

Kunci rahasia pada model simetris adalah kunci yang nilainya independen dari teks asli dan algoritma. Contoh implementasinya adalah Algoritma 4. Pada kode tersebut diketahui bahwa kunci yang sama dapat mendekripsi cipher text.

Note:
Perhatikan bahwa kriptografi klasik selalu menggunakan kunci simetris!

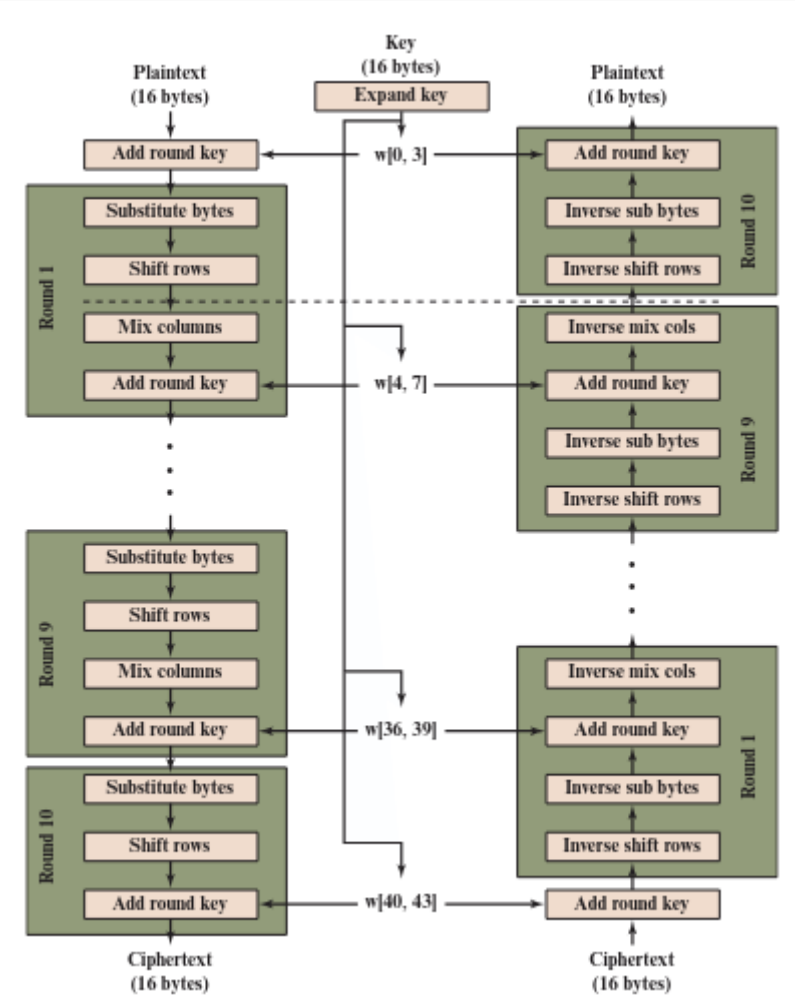


Gambar 4. *Kunci Simetris*
Sumber: Stallings, 2023

AES

Overview

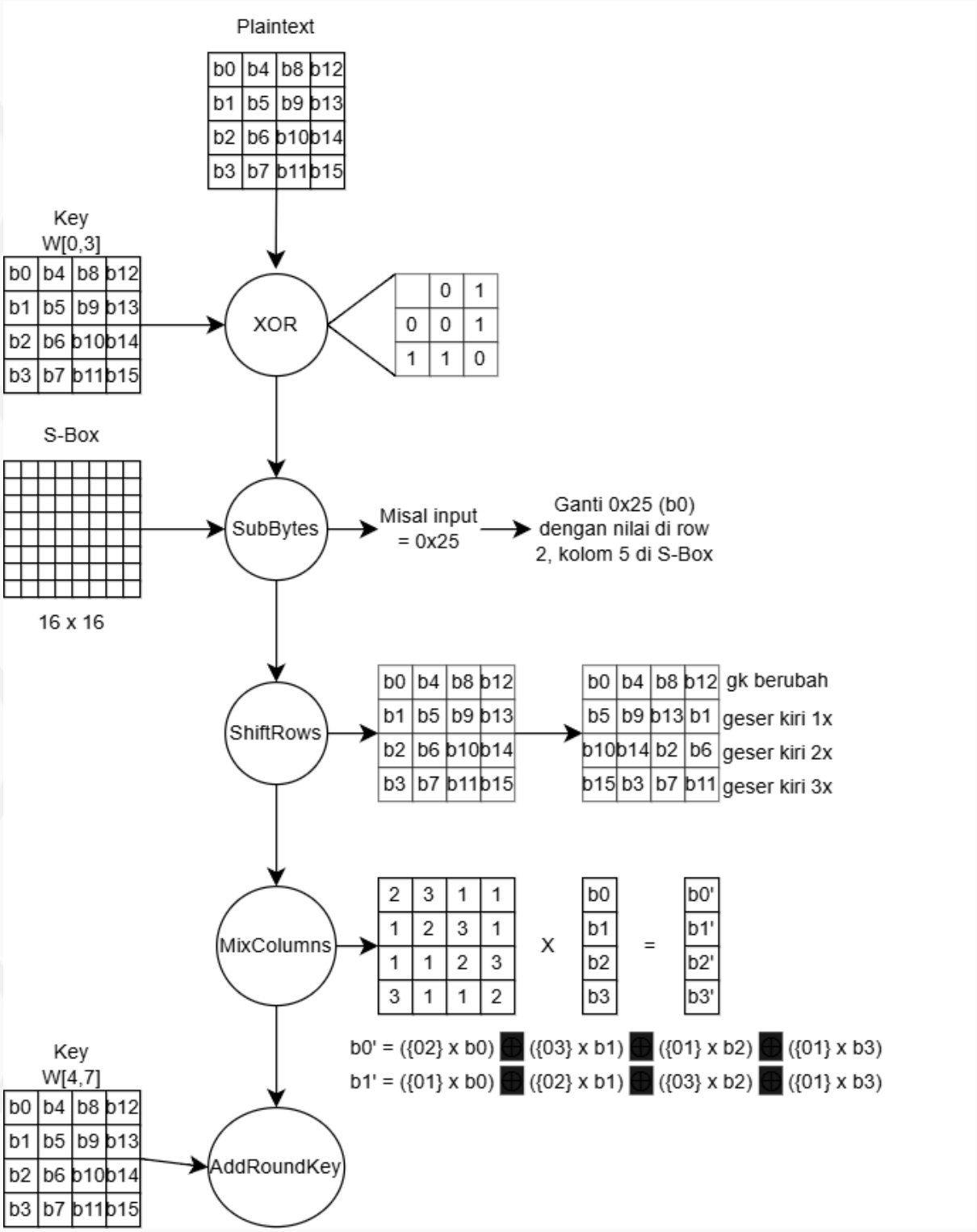
Algoritma Rijndael(dibaca: Rhine-doll) adalah algoritma yang ditetapkan sebagai AES (Advanced Encryption Standard) oleh NIST pada bulan Oktober 2000(Munir, 2019). Garis besar algoritmanya digambar pada bagian kiri diagram pada Gambar 5.



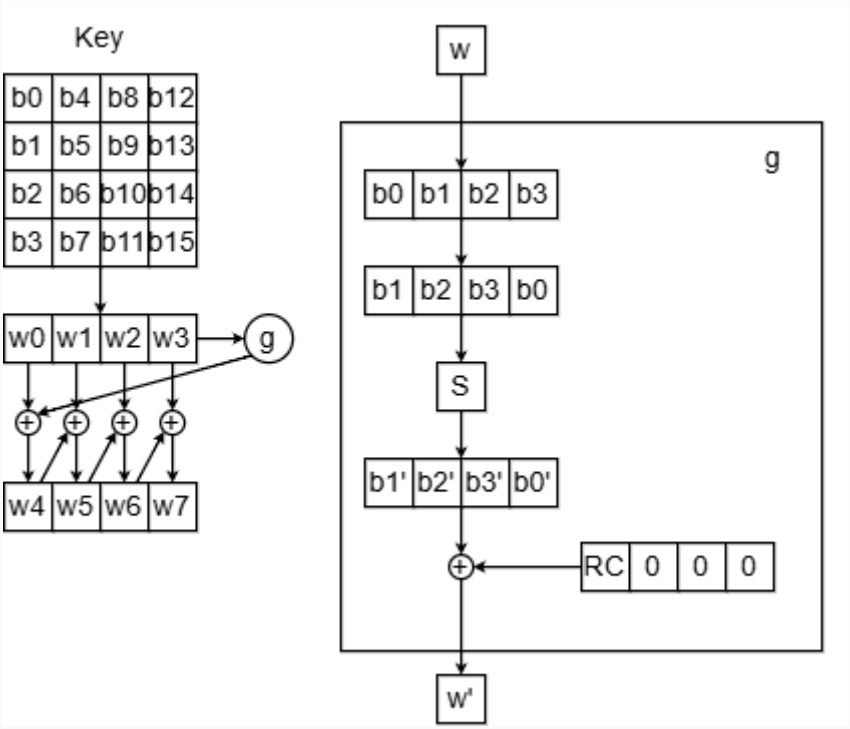
Gambar 5. *Algoritma AES*
Sumber: Stallings, 2023

Penjelesan.

1. Plain text di-passing ke AddRoundKey.
2. AddRoundKey melakukan XOR antara state awal(plain text) dengan cipher key.
3. Putaran sebanyak Nr-1 kali. Karena Nr = 10, Nr-1 = 9. Proses yang dilakukan setiap putaran adalah.
    a. SubBytes: substitusi byte dengan menggunakan tabel substitusi(S-box).
    b. ShiftRows: pergeseran baris-baris array state secara wrapping.
    c. MixCollumns: mengacak data di masing-masing kolom array state.
    d. AddRoundKey.
4. Proses diputaran terakhir
    a. SubBytes
    b. ShiftRows
    c. AddRoundKey
5. Hasil yang didapat adalah cipher text yang dinyatakan dalam state.



Gambar 6. *Diagram penjelasan AES*
Sumber: Yours truly



Gambar 7. *Diagram penjelasan ExpandKey*
Sumber: Yours truly

Implementasi

Implementasi menggunakan library open source. Ikuti langkah-langkah berikut.

1. Instal library pyaes melalui pip: [pip install pyaes]
2. Buatkan file project sebagai berikut
   (project name)/
   └── main.py
3. Masukkan kode yang ada pada Algoritma 5 ke dalam main.py!
4. Jalankan dengan: python main.py

Algoritma 5. *Implementasi AES*

```
import pyaes

def print_hex(label, data):
    hex_str = ' '.join(f'{b:02x}' for b in data)
    print(f"{label}: {hex_str}")

if __name__ == "__main__":
    # 1. Kunci 128-bit (16 Karakter)
    key = b'mysecretkey12345'

    # 2. Data yang mau dienkripsi (Wajib 16 byte untuk 1 blok ECB)
    plaintext = b'Halo Fufu Fafa!\x00'

    print("═══ PROSES AES-128 ECB ═══")
    print(f"Data Asli : {plaintext.decode(errors='replace')}")

    # 3. ENKRIPSI
    # Inisialisasi AES ECB encryptor
    aes_enc = pyaes.AESModeOfOperationECB(key)
    ciphertext = aes_enc.encrypt(plaintext)

    print_hex("Terenskripsi", ciphertext)
```

```
# 4. DEKRIPSI
aes_dec = pyaes.AESModeOfOperationECB(key)
decrypted = aes_dec.decrypt(ciphertext)

print(f"Hasil Kembali: {decrypted.decode(errors='replace')}")
print("==========================")
```

Challenge!
Ubahlah main.cpp untuk mengenkripsi file!

Note: main.py cuk bkn main.cpp

Tugas

1. Jalankan Algoritma 5 (implementasi AES) dengan mengganti kunci dan plaintext menggunakan data Anda sendiri. Kunci harus tetap 16 byte (16 karakter) dan plaintext juga harus tepat 16 byte. Lakukan proses enkripsi lalu dekripsi, dan pastikan hasil dekripsi sama dengan plaintext awal. Screenshot output program Anda.
   Output format: Source code Python dan screenshot output dalam .pdf
2. Buka file aes.py dari library pyaes (lokasi: lihat hasil perintah "pip show pyaes" pada bagian Location, lalu buka folder pyaes/aes.py). Pada tabel S-Box (array sbox), ganti nilai pada posisi pertama (indeks 0) dari 0x63 menjadi 0x00. Simpan perubahan, lalu jalankan kembali program dengan plaintext dan kunci yang sama seperti nomor 1. Bandingkan ciphertext sebelum dan sesudah modifikasi S-Box. Apakah hasilnya berbeda? Jelaskan secara singkat mengapa perubahan satu nilai saja pada S-Box dapat mempengaruhi output.
   Output format: Screenshot perbandingan output (sebelum dan sesudah modifikasi) dan penjelasan singkat dalam .pdf
3. Kembalikan S-Box ke semula (undo modifikasi nomor 2). Kemudian, ubah jumlah putaran AES dari 10 menjadi 5 dengan mengubah nilai Nr pada file aes.py dari library pyaes (cari variabel yang mengatur jumlah round, biasanya terkait dengan panjang kunci). Jalankan program dengan plaintext dan kunci yang sama. Bandingkan ciphertext hasil putaran 10 (asli) dan putaran 5 (modifikasi). Jelaskan secara singkat apa dampak pengurangan jumlah putaran terhadap keamanan enkripsi.
   Output format: Screenshot perbandingan output dan penjelasan singkat dalam .pdf

Catatan:

1. **Nomor 1 berupa source code Python terpisah dan screenshot output program dalam .pdf**
2. **Nomor 2 dan 3 berupa screenshot dan penjelasan dalam .pdf**
3. **Semua file dimasukkan ke dalam .ZIP**

Link Pengumpulan

Deadline:
C1: 12/04/26 23:59 WIB
C2: 13/04/26 23:59 WIB

Daftar Pustaka

Munir, R. (2019). *Kriptografi*. Informatika.

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (Eighth edition, global edition). Pearson.

Tim Asisten Praktikum

1. Franklin
2. Lyan
3. **Ichsan (PIC Praktikum)**
4. Alvin
5. Radhi